

Explainable Machine Learning for Effective Malware Detection in Encrypted Network Traffic

Joel C. Okore
Faculty of Computer Science
Innopolis University
Innopolis, Tatarstan, Russia
j.okore@innopolis.university

Isaac Womoakor
Faculty of Computer Science
Innopolis University
Innopolis, Tatarstan, Russia
i.womoakor@innopolis.university

Igor V. Kotenko
Security Problems Lab
SPC RAS
St. Petersburg, Russia
ivkote@comsec.spb.ru
Institute of Information Security
Innopolis University
Innopolis, Tatarstan, Russia
i.kotenko@innopolis.ru

Abstract—Transport Layer Security is the backbone of secure web communication today. It was designed to provide confidentiality, integrity, and authentication on untrusted networks, such as the Internet. With encryption becoming an essential component in protecting the integrity and confidentiality of data, it simultaneously makes the process of intrusion detection more difficult. Although machine learning approaches have demonstrated impressive accuracy in detecting malicious encrypted traffic using flow-level metadata, their black-box nature restricts operational trust and seamless integration into Security Orchestration, Automation, and Response environments. In this work, an explainable machine learning framework for the detection of malicious encrypted traffic is proposed using metadata-derived flow statistics and temporal patterns. We employ multiple feature-selection strategies, including Analysis of Variance statistical filtering and Shapley Additive Explanation importance ranking, on tree-based models to investigate the trade-off between detection performance and interpretability. The experimental results show that feature reduction supported by statistical validations not only preserves competitive accuracy but also improves explanation consistency and operational efficiency. The paper concludes by exploring how explainability improves trust in security automation workflows by translating model decisions into clear and actionable insights.

Index Terms—Encrypted Traffic Analysis, Explainable Artificial Intelligence, Malware Detection, Flow-based Intrusion Detection, Shapley Additive Explanation, Feature Selection, Security Automation.

I. INTRODUCTION

Malware represents a persistent cyber threat and with the widespread adoption of encryption in network communications, it is increasingly being transmitted over encrypted channels. As organizations progressively rely on encryption protocols such as TLS/SSL (Transport Layer Security/Secure Sockets Layer) to secure their networks, malware has evolved to exploit these protocols to conceal malicious activity within encrypted traffic. Machine learning (ML) approaches continue to demonstrate outstanding performance in detecting encrypted malicious traffic using flow-based statistical features. Previous studies report remarkable accuracy across tree-based models, ensemble methods, and deep learning architectures when trained on flow-based engineered statistics. Due to the black-

box nature of most ML-based intrusion detection systems, security analysts are left without clear insight into their decision-making processes, which undermines operational trust and complicates integration into Security Orchestration, Automation and Response (SOAR) platforms.

Explainable Artificial Intelligence (XAI) provides tools for interpreting the decisions made by ML models. XAI techniques can be further classified into intrinsic explainability (for example, "glass-box" models like Explainable Boosting Machines, linear models, and decision trees) and post-hoc explainability, for example, "black-box" techniques such as Shapley Additive Explanation (SHAP) and Local Interpretable Model-agnostic Explanations (LIME). Integrating XAI into cybersecurity environments introduces several challenges, including the need for explanations to be computationally efficient, consistent across executions, and truly reflective of the model's internal reasoning. These operational constraints are rarely considered, let alone quantified, in research on detecting malicious encrypted traffic.

To remedy these gaps, this work proposes an explainable ML framework for the detection of malicious encrypted traffic based exclusively on flow-level metadata. We design and empirically evaluate a two-tier explainability pipeline, with tier one being a fast and always-on explanation mechanism that is suitable for real-time Security Operations Center (SOC) triage and the second a high-fidelity, on-demand explanation module intended for forensic investigation. The paper then proposes how explainable outputs can be mapped to interpretable and actionable security insights, supporting trust-aware SOAR workflows.

II. RELATED WORK

With the increasing adoption of encryption protocols such as TLS/SSL, detecting malicious activity in encrypted traffic poses a significant research challenge. Early work has shown that contextual flow metadata and TLS handshake information prove useful in effectively identifying malicious from benign encrypted traffic without the need for payload decryption [1].

Subsequent studies extended the approaches to feature engineering by including statistical, temporal (time-based), and multi-view characteristics of encrypted flows, thus achieving strong performance with ensemble learning models [2], [3]. Comprehensive surveys further report the shift from Deep Packet Inspection (DPI) to metadata-driven ML techniques, clearly demonstrating the effectiveness of flow-based representations in the capture of network communication dynamics [4], [5].

XAI has emerged as a very important complementary field. It provides tools to improve transparency in ML-based cybersecurity systems. In general, XAI taxonomies distinguish between model-agnostic and model-specific explanation techniques, as well as global versus local interpretability mechanisms and their trade-offs [6]. Within encrypted malicious traffic detection, SHAP-based feature attribution has been integrated into ensemble classifiers to identify influential TLS and statistical features [7]. These works demonstrate the feasibility of applying feature attribution methods to encrypted traffic classification; however, they largely focus on insight into models' explanations rather than real-world constraints, such as operational stability, explanation latency, or integration into automated response pipelines. Addressing these gaps is critical for bridging high-performing encrypted traffic classifiers with automation-ready security operations workflows.

III. METHODOLOGY AND IMPLEMENTATION

A. Dataset and Feature Engineering

We use the Composed Encrypted Malicious Traffic Dataset proposed by [8], which integrates five public datasets: the Malware Capture Facility Project Dataset, CICIDS-2012, CIC-AndMal 2017, CICIDS-2017, and UNSW-NS 2019. The dataset was constructed to provide a class-balanced benchmark for the detection of encrypted malicious traffic, ensuring approximately equal contributions from each source dataset and a near-equal distribution of malicious and legitimate traffic. It consists of feature-based flow records extracted from preprocessed PCAP files, covering both conventional device and IoT encrypted traffic. The final dataset contains 117,627 sessions (59,516 malicious and 58,111 legitimate), enabling robust supervised learning without requiring additional resampling techniques. The raw flow records contain 113 features. We first remove all identity and leakage-prone columns, specifically source and destination IP addresses, port numbers, session identifiers, raw byte counters that encode dataset scale, and variance features redundant with their standard deviation counterparts. This leaves 89 candidate features. We then define a feature set termed "realtime-safe", consisting exclusively of per-packet measurements and windowed session statistics that can be computed without buffering the entire flow session: packet length distributions, TCP window statistics, inter-arrival times (forward and backward), time-to-live distributions, payload change counts, and TCP window change counts. This constraint ensures that all features are computable in an operational streaming context, which is a prerequisite for deployment in real-time SOC environments.

B. Detection Models

We train and evaluate three classifier families:

RF is an ensemble of 600 decision trees with square-root feature sampling at each split. RF provides naturally interpretable global feature importances and serves as a strong baseline ensemble.

XGBoost uses gradient boosting with 600 estimators, max depth 9, learning rate 0.1, and L1/L2 regularisation ($\alpha = 0.001$, $\lambda = 0.5$). XGBoost exposes built-in per-instance feature contributions via its prediction API, making it the primary Tier-1 black-box model in our explainability pipeline.

EBM is a glass-box generalised additive model trained using the InterpretML framework. EBM is inherently interpretable by design — its additive structure guarantees that each feature's contribution to any prediction is exactly recoverable without approximation. We include three-way interaction terms and train on a 100,000-instance random subsample of the training set to maintain tractable training time. EBM serves as our Tier-1 intrinsic explainer.

Hyperparameters for RF and XGBoost were tuned using five-fold stratified cross-validated random search optimizing macro F1-score. A stratified 80/20 train-test split with a fixed random seed (42) was used for reproducibility. The RF model employs 600 trees with unlimited depth and square-root feature selection. XGBoost is configured with 600 estimators, max depth 9, learning rate 0.1, subsample 0.7, full column sampling, and regularization parameters $\alpha = 0.001$, $\lambda = 0.5$, and $\gamma = 0.1$. For EBM, we employed up to 3-way feature interactions, trained on a subset of 100,000 samples, with a fixed random state of 0.

C. Feature Selection Strategies

We evaluated four feature subsets to assess the trade-off between feature count, detection performance, and explanation consistency: Features Full (65 features): all realtime-safe features after leakage exclusion; Features ANOVA 20 (20 features): top-20 features ranked by one-way ANOVA; Features SHAP 20 (20 features): top-20 features ranked by mean value computed via TreeSHAP on 2,000 held-out test instances; Features Intersection (10 features): features appearing in both the ANOVA top-20 and SHAP top-20 rankings, representing a consensus subset agreed upon by both statistical and model-driven selection. Additionally, we derive *Best Features* (9 features) as the intersection of the top-20 importance rankings from Random Forest and XGBoost independently, representing the minimal high-confidence subset.

D. Two-Tier Explainability Pipeline

The central contribution of this work is a two-tier explainability pipeline designed around the operational constraints of a SOC environment, where explanation latency and consistency directly affect throughput and trust.

Tier 1 (Always-On, Fast) executes for every classified flow and must complete within single-digit to tens of milliseconds. We implement two Tier-1 explainers: 1) XGBoost built-in

contributions: XGBoost’s booster API exposes exact per-feature SHAP-style contributions computed via tree traversal in $O(\text{depth})$ time per tree, without any sampling or approximation. 2) EBM local explanations: Since the EBM is an additive model, its local explanation is exact by construction — the contribution of the feature i to the prediction $f(x)$ is simply the learned shape function $g_i(x_i)$ evaluated at x_i .

Tier 2 (On-Demand, High-fidelity) executes only for flagged flows — those where $P(\text{malicious}) \geq 0.80$ or where the model is uncertain ($P(\text{malicious}) \in [0.45, 0.55]$). Two Tier-2 explainers are implemented: 1) LIME: a local surrogate model fitted via perturbed sampling (2,000 samples per instance). LIME is stochastic, making it suitable for measuring explanation stability. 2) SHAP TreeExplainer: exact Shapley values computed using the TreeExplainer algorithm, used as a reference to validate Tier-1 explanation fidelity.

All explainers expose a unified `explain_instance()` API that returns a standardized explanation record containing: instance identifier, model name, tier, predicted label, prediction confidence, top- k feature attributions (feature name, value, signed contribution, direction), and wall-clock explanation time in milliseconds.

E. Evaluation Metrics

Explanation Efficiency is measured as per-instance latency (p50 and p95 in milliseconds) and throughput (explanations per second), calculated over 1,000 held-out test instances for Tier-1 methods and 100 instances for Tier-2 LIME.

Explanation Stability is measured via top- k Jaccard similarity. For each of 20 test instances, LIME is run 10 times with different random seeds, and the Jaccard similarity between the first run’s top-5 feature set and each subsequent run is recorded. XGBoost contributions are run twice as a deterministic baseline.

Feature Selection Impact is measured by retraining RF and XGBoost on each feature subset and comparing accuracy, F1, and ROC-AUC on the held-out test set.

IV. RESULTS AND DISCUSSION

A. Detection Performance

Table I summarises classifier performance on the 65-feature realtime-safe feature set. All three models achieve near-perfect detection, confirming that metadata-only flow features are highly discriminative for this task without requiring payload decryption. The glass-box EBM achieves performance within 0.09% of the best black-box model (Random Forest), demonstrating that interpretability does not need to come at a meaningful accuracy cost for this task.

TABLE I
MODEL PERFORMANCE ON *Features Full* (65 FEATURES)

Model	Accuracy	F1	ROC-AUC
Random Forest	0.9998	0.9997	1.0000
XGBoost	0.9997	0.9997	1.0000
EBM	0.9989	0.9989	0.9998

For reference, baseline comparisons on the full 89-feature set show that the larger model family spanning Logistic Regression (F1: 0.801), SVM-RBF (F1: 0.877) and k -NN (F1: 0.871) perform substantially worse, confirming that tree-based ensembles are the appropriate choice. Gaussian Naive Bayes (F1: 0.730) performs worst, likely due to its strong independence assumption failing on correlated flow statistics.

B. Feature Selection Results

Table II reports the impact of feature subset choice on the detection performance.

TABLE II
FEATURES SELECTION COMPARISON

Features (F.) Set	F	RF F1	XGB F1	RF AUC	XGB AUC
<i>F. Full</i>	65	0.9998	0.9997	1.0000	1.0000
<i>F. SHAP 20</i>	20	1.0000	0.9989	1.0000	1.0000
<i>F. ANOVA 20</i>	20	0.9981	0.9922	1.0000	0.9997
<i>F. Intersection</i>	10	0.9976	0.9936	1.0000	0.9998
<i>Best Features</i>	9	0.9999	0.9997	1.0000	1.0000

Two observations are notable. First, *Features SHAP 20*, with only 20 features, matches or marginally exceeds *Features Full* for RF (F1: 1.0000 vs. 0.9998), demonstrating that model-driven feature selection not only preserves accuracy but can improve it by removing noisy features. Second, the 9-feature *Best Features* subset — defined as the intersection of RF and XGBoost top-20 importance rankings — retains near-perfect performance (RF F1: 0.9999), confirming that the detection signal is concentrated in a compact, semantically meaningful set of timing, window, and payload-change features. The most influential features identified by XGBoost include standard deviation of time-to-live (TTL), backward inter-arrival time statistics, payload change frequency, TCP window size, and packet timing characteristics.

C. Two-Tier Pipeline: Efficiency

Table III reports explanation latency measured over 1,000 test instances (100 for LIME).

TABLE III
EXPLANATION LATENCY

Method	Tier	p50 (ms)	p95 (ms)	Throughput (exp/s)
EBM (Local)	1	3.5	3.6	285
XGBoost Contributions	1	12.7	13.2	76
LIME	2	6.2	6.8	157
SHAP TreeExplainer	2	12.8	15.6	74

EBM is the fastest explainer at 3.5 ms p50 and 285 explanations per second, approximately $3.7\times$ faster than SHAP and $1.8\times$ faster than LIME. This is expected given EBM’s additive structure — local explanation reduces to a direct shape-function lookup per feature, with no sampling or tree-traversal overhead. XGBoost contributions and SHAP TreeExplainer exhibit near-identical latency (12.7 ms vs. 12.8 ms p50) and throughput (76 vs. 74 exp/s), confirming that both rely on the same underlying exact tree-traversal mechanism.

LIME, despite being a Tier-2 method, achieves lower median latency (6.2 ms p50) than XGBoost contributions in this configuration.

D. Two-Tier Pipeline: Stability

Table IV reports top-5 Jaccard stability across 10 repeated explanations for 20 test instances.

TABLE IV
EXPLANATION STABILITY (TOP-5 JACCARD)

Method	Mean Jaccard	Std	Min
XGBoost Contributions	1.000	0.000	1.000
LIME	0.899	0.137	0.655

XGBoost contributions are perfectly stable (Jaccard = 1.0 across all runs and all 20 instances), as expected for a deterministic tree-traversal algorithm. LIME achieves a mean Jaccard of 0.899 ± 0.137 , indicating that on average approximately 4.5 of the top 5 features are consistently identified in repeated runs. The minimum observed Jaccard of 0.655 corresponds to instances near the decision boundary where LIME’s local approximation neighbourhood is less constrained, producing higher sensitivity to the random perturbation seed. This instability directly motivates the two-tier design. Tier-1 handles the large majority of flows with deterministic, zero-variance explanations suitable for automated SOC alert annotation. Tier-2 LIME is reserved for forensic investigation contexts where an analyst can interpret and account for stochasticity.

E. Security Insights into Top Features

The features most consistently ranked across the SHAP, ANOVA, and Tier-1 XGBoost contributions map directly onto interpretable security semantics. Five of the nine *Best Features* are inter-arrival time (IAT) or timing statistics, confirming that temporal regularity is the dominant discriminative axis, which is consistent with prior work on TLS fingerprinting.

Std Time To Live, the single highest SHAP-ranked feature, captures operating-system fingerprint uniformity: malware often originates from fixed environments with rigid TTL initialisation, whereas benign traffic reflects heterogeneous endpoints. *The Times of Change of Payload Per Session* serves as a proxy for application-layer content diversity — automated command-and-control (C2) sessions exchange fixed-format messages, producing fewer payload-size transitions than a browser loading mixed content. *Mean TCP Windows Size Value* reflects programmatic versus organic connection behaviour, as malware connections tend to maintain fixed window sizes, while browsers dynamically adjust in response to application-layer backpressure.

These mappings enable Tier-1 output to be rendered as analyst-readable SOAR annotations. For example, a Tier-1 explanation identifying a high contribution from *Std Time To Live* alongside a low value of *Std Interval of Arrival Time of Backward Traffic* can be rendered as: “**ALERT:** Session flagged due to high variability in packet TTL values combined with low variability (high consistency) in server response intervals. This pattern suggests automated periodic communication

occurring over heterogeneous infrastructure, consistent with potential C2 beaconing behavior.” This kind of semantically grounded annotation gives the analyst a verifiable rationale rather than an opaque confidence score, directly supporting trust-aware SOAR automations.

V. CONCLUSION

We presented an explainable ML framework to detect malicious encrypted traffic using metadata-only flow statistics. Our two-tier explainability pipeline achieves near-perfect detection ($F1 \geq 0.9989$) in Random Forest, XGBoost, and the glass-box EBM, with the EBM within 0.09% of the best black-box model. EBM local explanations are the most efficient Tier-1 mechanism at 3.5 ms p50 (285 exp/s), approximately $3.7\times$ faster than SHAP. LIME achieves 0.899 mean Jaccard stability, making it suitable for forensic investigation contexts. Feature selection experiments demonstrate that SHAP-ranked reduction to 20 features matches or exceeds full-feature performance and that a 9-feature consensus subset retains near-perfect accuracy. Future work will evaluate generalization to live traffic and explore amortized explanation methods to reduce Tier-2 latency.

VI. ACKNOWLEDGMENT

The first two authors greatly appreciate the mentorship of their MSc project supervisor - Dr Andrei Petrovski - in the process of writing the paper. The reported study was funded by the grant of the St. Petersburg Science Foundation No. 25-PB-01-13.

REFERENCES

- [1] B. Anderson, S. Paul, and D. McGrew, “Deciphering malware’s use of tls (without decryption),” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 12, pp. 3216–3230, 2017, accessed: Nov. 30, 2025. [Online]. Available: <https://link.springer.com/content/pdf/10.1007/s11416-017-0306-6.pdf>.
- [2] A. S. Shekhawat, F. D. Troia, and M. Stamp, “Feature analysis of encrypted malicious traffic,” *Expert Systems with Applications*, vol. 125, pp. 130–141, 2019, accessed: Dec. 8, 2025. [Online]. Available: <https://doi.org/10.1016/j.eswa.2019.01.064>.
- [3] K. Jaspin, Y. S. Shinanth, and S. K. Kishore, “Real-time network threat detection using machine learning and kafka in middleware,” in *Proceedings of the International Conference on Computational Intelligence and Data Science (ICCIDS)*, 2024, pp. 1–7, accessed: Dec. 9, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/10867200>.
- [4] K. Keshkeh, A. Jantan, K. Aliyan, and U. M. Gana, “A review on tls encryption malware detection: Tls features, machine learning usage, and future directions,” *Journal of Network and Computer Applications*, vol. 242, pp. 213–229, 2022, accessed: Nov. 30, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1084804524001933>.
- [5] Y. Li, H. Guo, J. Hou, Z. Zhang, T. Jiang, and Z. Liu, “A survey of encrypted malicious traffic detection,” *IEEE Communications Surveys Tutorials*, vol. 23, no. 3, pp. 1531–1566, 2021, accessed: Dec. 8, 2025. [Online]. Available: <https://doi.org/10.1109/CCCI52664.2021.9583191>.
- [6] A. V. J. B. Nugraha1 and T. Bauschert1, “A versatile xai-based framework for efficient and explainable intrusion detection systems,” *Annals of Telecommunications*, 2025.
- [7] A. Shahraki, M. Abbasi, A. Taherkordi, and A. D. Jurcut, “A comparative study on online machine learning techniques for network traffic streams,” *Computer Networks*, vol. 207, p. 108836, 2022, accessed: Dec. 9, 2025. [Online]. Available: <https://doi.org/10.1016/j.comnet.2022.108836>.
- [8] Z. Wang, K.-W. Fok, and V. L. L. Thing, “Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study,” *Computer Security*, vol. 113, no. 102542, p. 109132, 2022, accessed: Dec. 8, 2025. [Online]. Available: <https://arxiv.org/pdf/2203.09332>.